

SAKTHIVEL BALAKUMAR – 63957

MEKA VENKATA SRAVANTH BABU – 63919

SAMSUDEEN SABEER RAHMAN - 63920

LAB3 – INTEGRATE COMPONENTS

**Q1) Read the Verilog part of the component and explain the role of :
Cpu_ChipSel, Cpu_Write, Cpu_Addrs, Cpu_DataWr as part of the Avalon bus.**

Ans :

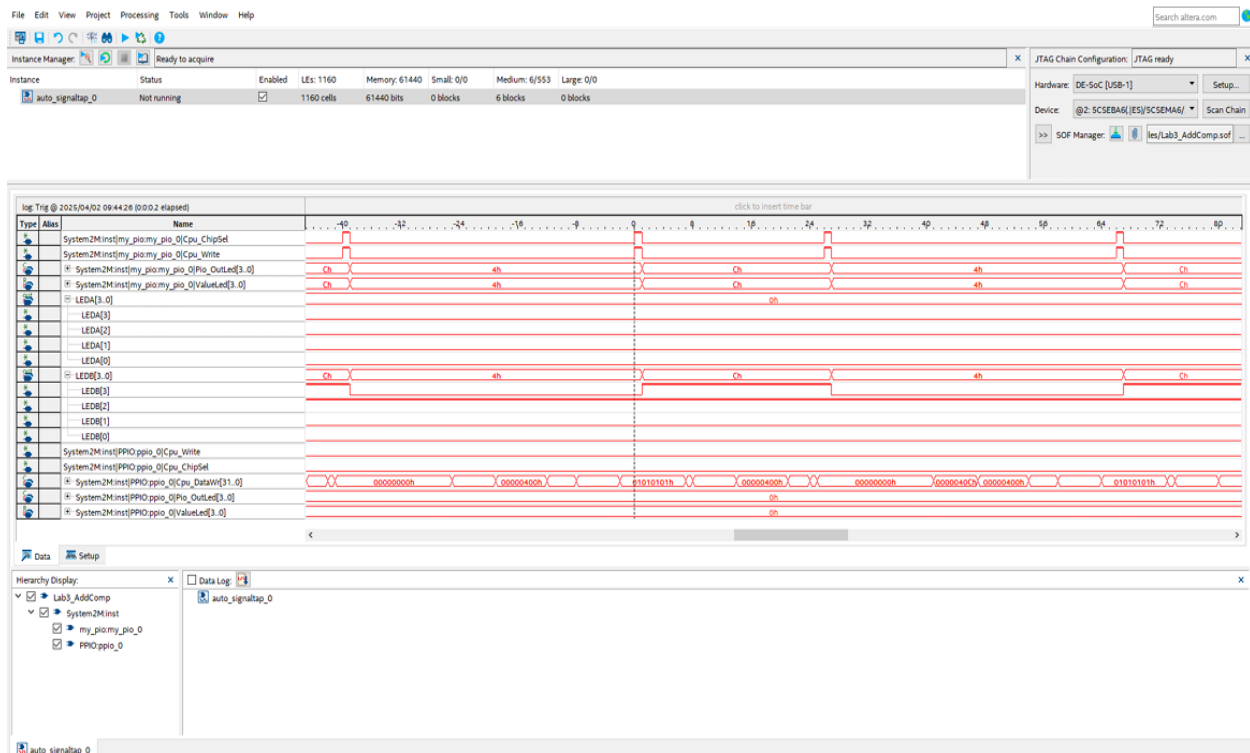
Cpu_ChipSel (chip select): Its role is to activate the peripheral (my_pio component) .When this signal is high(1), it means that the CPU is addressing the component.

Cpu_Write (Write Enable): It indicates a write operation. It goes high(1) when CPU wants to send data to the component. In Verilog it is often used to control when to latch Cpu_DataWr.

Cpu_Addrs (Address): Its role is to choose which internal register to access inside our module. It is decoded in Verilog to match a specific register or memory location.

Cpu_DataWr (Data Write): This component is the actual data from CPU that will be written into our my_pio component. It is captured on the rising edge of the clock when both Cpu_Write and Cpu_ChipSel are high.

Q2) Trap the Avalon data bus and the Address bus when you write data when your program is running. (Provide the chronogram in your report).



Q3) In your idea why do you have to write at the address “*(pled + 8)” to access this led ?

Ans:

In `*(pled + 8) = 0xFF`; The reason to write like this is that means the LED we are trying to control is mapped to the 8th byte offset from the base address of the MY_PIO_0 component. When the code writes to `*(pled + 8)`, it is used to access the memory address of 8 bytes after the base address.

This is very important because the LED you are trying to control is mapped to an internal register that is located 8 bytes away from the base of the PIO component. These address offsets (+0, +4, +8, +12) are determined by how the internal registers of the PIO are arranged in the hardware configuration. To access and control a specific LED, we must write to the memory address base + 8, which is done using `*(pled + 8)`.

Q4) Modify the C code to toggle the other LEDs:

Ans:

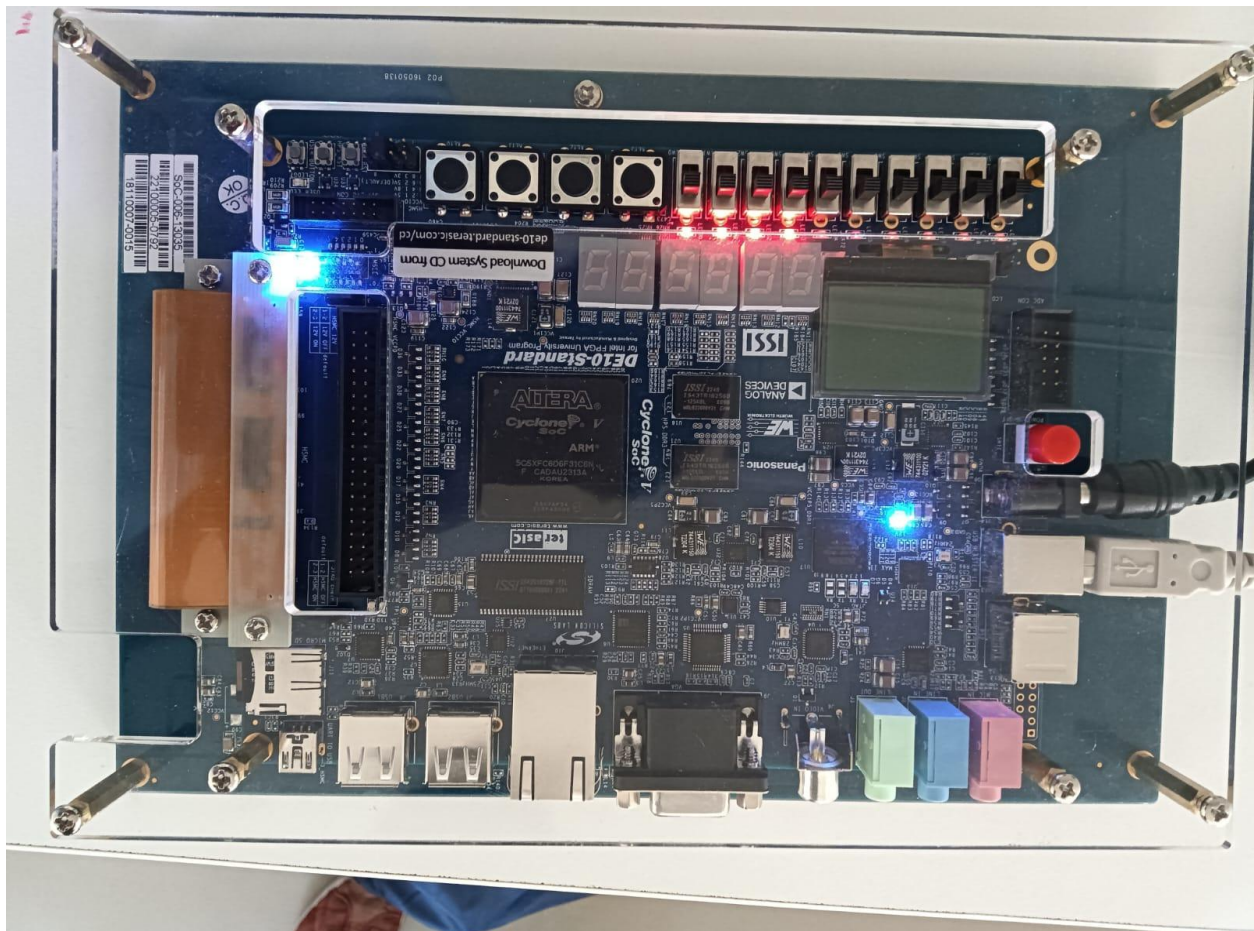
```
//-----  
//  
// LED Blinking Program for Nios II - All LEDs Blink  
//  
//-----  
  
#include <stdio.h>  
#include "system.h"  
  
#define      NBR_TEMPO      800000  
  
void Tempo(void);  
  
int main() {  
    volatile char *pled;  
  
    printf("\nHello from Nios II (ALL_LED_BLINK_Program)\n\n");  
  
    pled = (volatile char *)MY_PIO_0_BASE;  
  
    while (1) {        // Infinite loop  
        *(pled + 0) = 0xFF;    // Turn ON all LEDs  
        *(pled + 4) = 0xFF;  
        *(pled + 8) = 0xFF;  
        *(pled + 12) = 0xFF;  
        Tempo();  
  
        *(pled + 0) = 0x00;    // Turn OFF all LEDs  
        *(pled + 4) = 0x00;  
        *(pled + 8) = 0x00;  
        *(pled + 12) = 0x00;  
        Tempo();  
    }  
  
    return 0;  
}
```

```

void Tempo(void) {
    int nbc = NBR_TEMPO;
    while (nbc > 0) {
        nbc--;
    }
}

```

Blinking of all LED's:



Q4) Propose a modification of the Verilog code. Modify “my_pio.v”, update System2M on Platform Designer and generate it. Update the System2M on Quartus, compile the system, program the board, and test your new design.

Ans:

Assuming that our original my_pio.v is a simple PIO (parallel I/O) module. We have to add an input to enable or disable the LEDs. Second step is to update in Platform Designer. Open the platform designer, remove the old my_pio instance. Then add new my_pio.v and create a new module from this file. Map the new port (led_enable) to a PIO switch or software control. Save the system as System2M. Then generate HDL. Then final step is to compile the program and test the updated c code.